

UNIVERSIDAD NACIONAL DE INGENIERÍA

Facultad de Ingeniería Industrial y de Sistemas · Escuela de Ingeniería de Software

SW407 · Sistemas Operativos

SEMANA 1: INTRODUCCIÓN A LOS SISTEMAS OPERATIVOS

¿Qué hace posible que el hardware cobre vida?

Objetivos de la sesión

1. Definir qué es un sistema operativo y explicar por qué es indispensable.
2. Reconocer los hitos históricos que llevaron del sistema por lotes a los SO modernos.
3. Distinguir los tipos de sistemas operativos según su forma de atender usuarios y tareas.
4. Identificar los tres bloques básicos de la arquitectura de un SO: núcleo, shell y servicios.

Pregunta guía: *Si compraras una laptop sin sistema operativo instalado, ¿qué podrías hacer con ella?*

El gerente invisible del hotel

> Piensa en un hotel de 200 habitaciones: llegan huéspedes (programas) que piden habitación (memoria), servicio de room service (impresora, disco), y acceso al ascensor (CPU) al mismo tiempo. Sin un gerente que reparta recursos, asigne prioridades y evite que dos huéspedes reclamen la misma habitación, el hotel colapsaría en minutos. El gerente nunca atiende directamente a los huéspedes: coordina al personal (hardware) para que cada solicitud se resuelva de forma ordenada, segura y eficiente.

En términos del SO: El sistema operativo es ese gerente: un software que administra CPU, memoria, dispositivos y archivos para que los programas de usuario puedan ejecutarse sin chocar entre sí ni con el hardware.

¿Qué es un sistema operativo?

DEFINICIÓN TÉCNICA

Un SO es el software base que se ejecuta apenas enciende el equipo. Actúa como intermediario entre el hardware (CPU, memoria, discos, periféricos) y los programas de aplicación, ofreciendo una interfaz uniforme y ocultando la complejidad del hardware.

EJEMPLO PRÁCTICO

Cuando abres una app de fotos en tu celular, tú solo tocas un ícono. Por debajo, el SO reserva memoria, carga el archivo desde el almacenamiento, programa la CPU para decodificar la imagen y dibuja el resultado en pantalla — todo en milisegundos, sin que lo notes.

Evolución histórica en cuatro escenas

Por lotes (1950s)

Los programas se agrupaban en 'lotes' en tarjetas perforadas. El operador los ejecutaba uno tras otro, sin interacción. La CPU quedaba ociosa esperando E/S.

Multiprogramación

Varios programas conviven en memoria; mientras uno espera E/S, otro usa la CPU. Nace la idea de aprovechar recursos ociosos.

Tiempo compartido

Cada usuario recibe pequeñas rebanadas de CPU en rotación tan rápida que siente el sistema como propio y exclusivo.

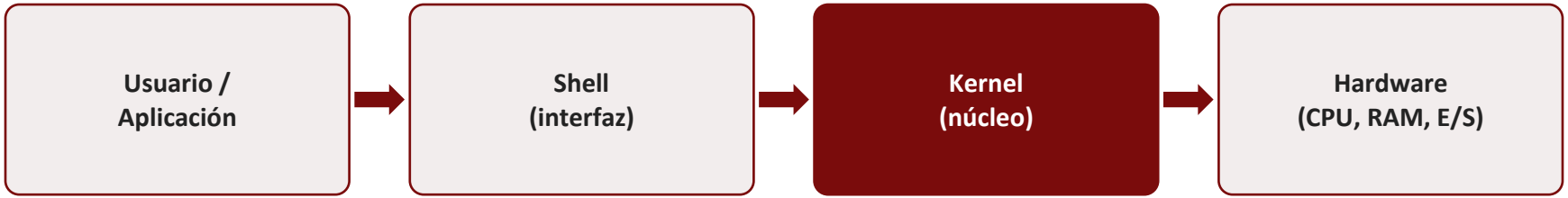
SO modernos

Multiusuario, multitarea, en red y distribuidos: Linux, Windows, macOS, Android — coordinan miles de procesos y usuarios simultáneos.

Tipos de sistemas operativos

Monousuario	Multiusuario	Tiempo real	Distribuido
Un solo usuario a la vez controla todos los recursos. Ejemplo: MS-DOS clásico, o tu celular personal en modo básico.	Varios usuarios comparten el mismo equipo simultáneamente. Ejemplo: un servidor Linux universitario con decenas de sesiones SSH activas.	Garantiza respuesta dentro de un plazo estricto. Ejemplo: el software que controla los airbags de un auto o una bomba de insulina.	Varios equipos independientes cooperan como si fueran uno solo. Ejemplo: los nodos de un clúster de Kubernetes.

Arquitectura básica de un SO



El kernel es el corazón del SO: gestiona procesos, memoria y dispositivos con acceso privilegiado al hardware. El shell es la puerta de entrada del usuario (línea de comandos o interfaz gráfica) que traduce sus órdenes en llamadas que el kernel entiende.

Actividad práctica

Trabajo en clase: SO en la vida cotidiana

- Discusión grupal: enumeren 5 dispositivos de su día a día que corren un SO (celular, smart TV, cajero automático, router, auto moderno).
- Para cada uno, identifiquen qué recurso administra el SO con mayor exigencia (CPU, memoria, batería, red).
- Instalen una máquina virtual con Linux (VirtualBox/VMware) y ejecuten su primer arranque para familiarizarse con el entorno.

Entregable: Cuadro comparativo de 5 dispositivos + captura de pantalla de la VM Linux funcionando.

Bibliografía de referencia

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- Stallings, W. (2001/2018). Sistemas Operativos (4ta ed. / 9th ed. internals). Pearson.
- Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th ed.). Pearson.
- Love, R. (2010). Linux Kernel Development (3rd ed.). Addison-Wesley.
- Biblioteca virtual UNI-FIIS. Recursos de Escuela de Ingeniería de Software sobre sistemas operativos.